11

Sessions

Plus **Beispieldateien:** Dateien aus Ordner *Kapitel_11*

11.1 Mit Sessions arbeiten

Grundlagen zu Sessions

Eine Datenübertragung im Internet erfolgt über das Protokoll HTTP. HTTP ist allerdings ein **zustandsloses Protokoll**, das heißt, mehrere Anfragen, also Seitenaufrufe – auch desselben Seitenbesuchers – sind grundsätzlich voneinander unabhängige Aktionen. Daten, die beispielsweise ein Benutzer in ein Formular eingegeben hat, werden in Variablen nur bis zum auswertenden PHP-Programm weitergegeben. Bereits auf der übernächsten aufgerufenen Webseite sind die Daten nicht mehr vorhanden. Der Webserver kann also, wenn keine weiteren Maßnahmen ergriffen wurden, **nicht** feststellen, welche Seitenaufrufe vom selben Benutzer kommen.

Für diverse Abwicklungen auf Webseiten benötigen Sie jedoch genau eine Funktionalität, über die Sie einen **Besucher wiedererkennen** und mit ihm verbundene Daten zwischenspeichern können. Sie wollen z. B. über mehrere Webseiten Eingaben ermöglichen und dann als Bestellung absenden (Shopsystem). Oder Sie möchten, dass sich auf Ihren Webseiten Benutzer einloggen können und möchten auf jeder weiteren Seite prüfen, ob der Benutzer durch seine Anmeldung autorisiert ist, die jeweilige Seite aufzurufen.

Gerade bei sogenannten personalisierten Seiten, die auf den Benutzer „reagieren“, ist es notwendig, bestimmte Daten, z. B. den Login-Status, von Seite zu Seite mitzuführen bzw. weiterzugeben.

PHP unterstützt diese Anforderungen mithilfe von Sessions (Sitzungen). Eine Session bietet die Möglichkeit, Daten während eines Webseitenbesuchs über mehrere Seiten zwischenzuspeichern und für jede einzelne Seite verfügbar zu halten. Der Nutzen einer Session liegt darin, dass während einer Session Daten gespeichert und diese von jeder weiteren PHP-Seite weiterverwendet werden können. Eine Session endet,

- wenn der Benutzer den Browser schließt,
- wenn eine Webseite mit entsprechenden Befehlen aufgerufen wird,
- nach einer definierten Zeitspanne
- oder – je nach Programmierung – auch beim Verlassen der Webseiten, auf denen die Session definiert wurde.

Schulversion
© HERDT-Verlag

Die Standarddauer einer Session beträgt 1440 Sekunden (**24 Minuten**). Mit jedem neuen Seitenaufruf während der Sitzung wird diese erneut gesetzt. Nach 1440 Sekunden Inaktivität wird die Sitzung des Seitenbesuches beendet. Zum Beenden einer Session reicht das Schließen eines einzelnen Tabs des Browsers nicht aus. Die Session wird erst beendet, wenn der **Browser komplett geschlossen** wird.

Session-IDs: Identifikation einer Session

Damit Informationen zu einem Besucher innerhalb einer Website seitenübergreifend verwaltet werden können, erhält ein Besucher beim ersten Zugriff auf mit Sessions definierte Webseiten eine zufällige, eindeutige 32-stellige ID, die **Session-ID**. Diese wird bei jedem Aufruf einer Seite an den Webserver gesendet, woran dieser den Besucher wiedererkennt. Anhand der Session-ID können gespeicherte Daten zugeordnet werden.

Solange der Besucher auf den Webseiten verweilt, auf denen die Session gestartet wurde, werden auf dem Webserver benutzerbezogene Daten in einer **Session-Datei** gespeichert. Dieses können z. B. Formulareingaben des Nutzers sein, aber auch Daten, die anhand von Login-Daten aus Datenbanken oder anderen Quellen ermittelt wurden. Bei der Session-Datei handelt es sich um eine Textdatei, die auf dem Webserver in einem Ordner für temporäre Dateien abgelegt wird. Bei der Standardeinstellung der im Buch verwendeten XAMPP-Installation handelt es sich unter Windows um das Verzeichnis *C:\xampp\tmp*. Unter macOS befindet sich das entsprechende Verzeichnis unter */lampp/temp (in der gemounteten virtuellen Maschine)*.

Es handelt sich hierbei um ein serverseitiges Session-Cookie. Cookie ist die Bezeichnung für die Textdatei, die Informationen über einen Vorgang sammelt. In jedem Skript, das zur Session gehört, wird die Session-ID zur abermaligen Authentifizierung benötigt. Die Übertragung der Session-ID von Skript zu Skript wird als **Durchschleifen** bezeichnet. Das Durchschleifen der Session-ID ist einfach zu realisieren, da diese Funktionalität intern von PHP verwaltet wird. Dazu werden die Daten der Session temporär auf dem Webserver gespeichert. Das Durchschleifen einer Session-ID findet nur innerhalb der PHP-Skripte statt, die eine Session mit der Funktion `session_start()` fortsetzen. Durch Aufruf einer Seite mit der Dateinamenerweiterung **.htm(l)* oder einer **.php*-Seite, auf der `session_start()` nicht aufgerufen wurde, verlässt der Nutzer also die Session.

Als Gegenstück zum Session-Cookie speichert der Browser einen Browser-Cookie mit der Session-ID ab. Die Session-ID wird mit jedem Seitenaufruf an den Server gesendet. Damit wird die konkrete Verknüpfung zwischen Client und Server hergestellt.

Falls Nutzer Cookies im Browser deaktiviert haben, funktionieren Sessions nicht korrekt. Theoretisch wäre es möglich, die Session-ID händisch von Seite zu Seite „durchzuschleifen“, indem man sie an alle Links anhängt und mit allen Formularen überträgt. Da aber im Grunde jede Webseite auf Session-Cookies basiert, ist das tatsächlich nur eine theoretische Überlegung.

Mit der Einführung der DSGVO in 2018 bekommen immer mehr Webseiten sogenannte Cookie-Banner, auf denen die Besucher über die Verwendung von Cookies informiert werden. Die DSGVO zielt vor allem auf Cookies ab, die Drittanbieter setzen, um Nutzerdaten, z. B. für Marketingzwecke, abzugreifen. Die durch Sessions gesetzten Cookies, wie in diesem Kapitel beschrieben, gehören zu den notwendigen Cookies einer Webseite. Sie gehören zu einer funktionierenden Webseite wie das Wasser zum Waschen.

© HERDT-Verlag
Schulversion

11.2 Session starten bzw. fortsetzen

Wenn Sie mit einer Session arbeiten wollen, gehen Sie wie folgt vor:

- Alle Dateien, die innerhalb der Session erreichbar sein sollen, müssen korrekte PHP-Dateien sein. Die Dateiendung muss **.php* lauten. Früher wurden Endungen wie **.php4* oder **.php5* verwendet, derweil hat sich **.php* als Standard etabliert, von dem man auch nicht abweichen sollte. Welche Dateiendung als PHP-Datei erkannt wird, ist in der *php.ini* konfiguriert.
- Jede dieser Dateien beginnt mit Öffnen eines PHP-Blocks (`<?php`) in der ersten Zeile der Datei, gefolgt von der Funktion `session_start()`, um eine Session zu starten bzw. eine laufende Session fortzusetzen. Das gilt auch für Dateien, die ausschließlich HTML-Tags enthalten.

Um eine Session zu starten, verwenden Sie die Funktion `session_start()`. Beim Aufruf des Programms wird eine Session gestartet und eine eindeutige Session-ID erzeugt. Auf die Session-ID können Sie bei Bedarf mit der Funktion `session_id()` zugreifen.

Syntax und Bedeutung der Funktion `session_start()`

```
session_start();
```

- Bei Aufruf der Funktion `session_start()` wird eine neue Session gestartet oder eine aktuelle Session wieder aufgenommen.

  Ist die vom User übermittelte Session-ID aktiv, wird diese ID von PHP automatisch erkannt und damit die bestehende Session fortgeführt. Wird keine Session-ID erkannt, generiert PHP eine neue Session-ID und startet damit eine neue Session.
- Jede Webseite, die zu einer Session gehören soll, muss den Befehl `session_start()` beinhalten, ansonsten kann keine Session gestartet bzw. fortgesetzt werden.
- Die Funktion erwartet keine Parameter.
- Rückgabewert der Funktion ist `TRUE` (bei Erfolg) bzw. `FALSE` (im Fehlerfall).



Sie müssen `session_start()`aufrufen, bevor Sie eine Ausgabe im Browser vornehmen. Eine `echo`-Anweisung in PHP oder ein HTML-Tag vor dem Aufruf dieser Funktion führen zu einer Fehlermeldung ("*header already sent*"). Denselben Effekt haben Leerzeichen außerhalb von PHP-Blöcken. Auch ein versehentlich eingefügtes Leerzeichen hinter dem schließenden PHP-Tag einer inkludierten PHP-Datei, die vor dem Aufruf von `session_start()` eingebunden wird, kann zu dieser Fehlermeldung führen. Um Schwierigkeiten zu vermeiden, sollten Sie als Erstes ganz oben in der Datei einen PHP-Block einfügen und danach `session_start()` aufrufen.

Standardmäßig lautet der Name der Session `PHPSESSID`, der in der Datei *php.ini* definiert ist. Sie können eine Session zur Laufzeit auch mit einem anderen Namen versehen. Über diesen Namen können Sie diese Session im späteren Verlauf wieder ansprechen.

© HERDT-Verlag

Syntax und Bedeutung der Funktion `session_name()`

Möchten Sie bei Aufruf einer Webseite einen eigenen Session-Namen definieren, rufen Sie die Funktion `session_name()` **vor** dem Starten der Session mit `session_start()` auf.

```
session_name(Bezeichnung);
```

Damit wird der Wert der Session mit dem angegebenen Namen erstellt bzw. übernommen.

Achtung: Der Parameter `Bezeichnung` in der Funktion `session_name ()` darf nicht nur aus Zahlen bestehen, sondern muss mindestens einen Buchstaben enthalten.

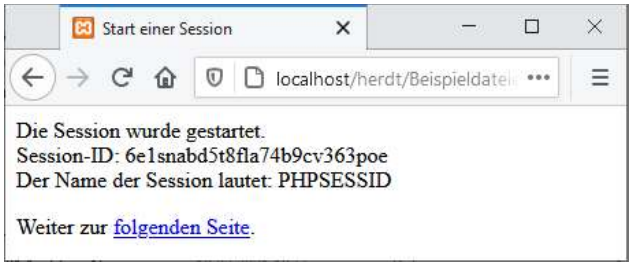
Beispiel *start.php*

```
<?php
① session_start();
② $id = session_id();
  echo "<!DOCTYPE html>";
  echo "<html><head>";
  echo "<meta charset=\"UTF-8\">";
  echo "<title>Start einer Session</title></head>";
  echo "<body>Die Session wurde gestartet.<br>";
  echo "Session-ID: " . $id;
③ echo "<br>Der Name der Session lautet: " . session_name();
  echo "<p>Weiter zur <a href='formular.php'>folgenden
        Seite</a>.</p>";
④ echo "</body></html>";
```

① Mit dem Befehl `session_start()` wird eine neue Session gestartet oder eine gestartete Session fortgesetzt.

② Der Variablen `$id` wird die aktuelle Session-ID zugewiesen.

③ Über die Funktion `session_name()` wird der Name der aktuellen Session ausgegeben.



Anzeige der Beispieldatei „start.php“

④ Da hier bereits die komplette HTML-Ausgabe geschehen ist und gerade im Zusammenhang mit Sessions auch die Ausgabe von Leerzeichen Probleme bereiten können, wird der schließende PHP-Tag `?>` weggelassen. Damit vermeiden Sie eine versehentliche Ausgabe von Leerzeichen am Dateiende. Die PHP-Datei und die Abarbeitung des Skripts enden damit.

11.3 Daten in einer Session speichern

PHP hilft Ihnen bei Start und Betrieb einer Session. Die automatisch angelegte Session-Datei kann gewünschte Informationen aufnehmen und für die Dauer der Session verfügbar halten. Das Speichern von Daten in einer Session hingegen geschieht nicht automatisch, sondern hängt von der Programmierung ab. Um Daten in der Session-Datei zu speichern, verwenden Sie die superglobale Array-Variable `$_SESSION`.

Mit dieser Zuweisung erreichen Sie, dass das Element unter dem Array-Schlüssel `"benutzer"` in die Session-Datei eingetragen wird.

```
$_SESSION["Element"] = Wert;
z.B. $_SESSION["benutzer"] = "Maxi";
```

Damit steht es in der Session als Element der Array-Variablen `$_SESSION` zur Verfügung. Am Ende eines PHP-Skripts werden üblicherweise alle Variablen aus dem Arbeitsspeicher des Servers gelöscht. So auch die Werte aus `$_SESSION`. Setzt das als nächstes aufgerufene Skript die Session fort, werden die Daten aus der Session-Datei in die Variable `$_SESSION` eingelesen. Die Inhalte stehen somit wieder zur Verfügung, nachdem Sie `session_start()` aufgerufen haben. Dieser Vorgang wiederholt sich von Skript zu Skript bis zum Ende der Session.

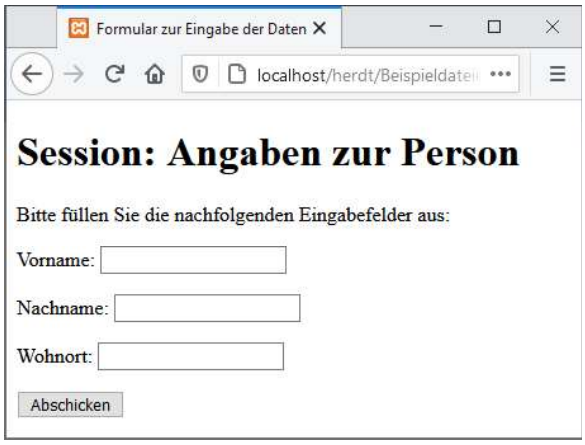
Beispiel: *formular.php*

In Beispiel wird ein Formular aufgebaut, in das Nutzer ihre Daten eingeben können und das nach Absenden des Submit-Buttons weiterverarbeitet wird. Im auswertenden Skript werden diese Daten an Variablen übergeben, auf die Sie innerhalb einer Session von jeder anderen Seite zugreifen können.

```
<?php
① session_start();
?><!DOCTYPE html>
<html>
  <head><meta charset="UTF-8">
     <title>Formular zur Eingabe der Daten</title></head>
  <body>
    <h1>Session: Angaben zur Person</h1>
    <p>Bitte füllen Sie die nachfolgenden Eingabefelder
    aus: </p>
② <form action="auswertung.php" method="POST">
      <p>Vorname: <input type="text" name="vorname"></p>
      <p>Nachname: <input type="text" name="nachname"></p>
      <p>Wohnort: <input type="text" name="ort"></p>
      <p><input type="submit" value="Abschicken"></p>
    </form>
  </body>
</html>
```


© HERDT-Verlag

① Eine Session wird initialisiert. In diesem Beispiel läuft bereits eine Session (falls Sie die Seite von der Seite *start.php* aufgerufen haben), also wird die bestehende Session fortgesetzt. Die Session muss lückenlos fortgesetzt werden, deshalb erhält auch die Datei, die nur ein HTML-Formular beinhaltet, die Dateinamenerweiterung **.php* und initialisiert eine Session. Da der PHP-Block zur Initialisierung der Session notwendig ist, muss es eine PHP-Datei sein. In HTML-Dateien können Sie diesen PHP-Block nicht einsetzen.



Ausgabe der Beispieldatei „formular.php“

② Ab hier wird ein einfaches Formular aufgebaut. Als Ziel-Skript des Formulars wird über das `action`-Attribut die PHP-Datei *auswertung.php* definiert, in der die Auswertung des Formulars vorgenommen werden soll.

Beispiel: *auswertung.php*

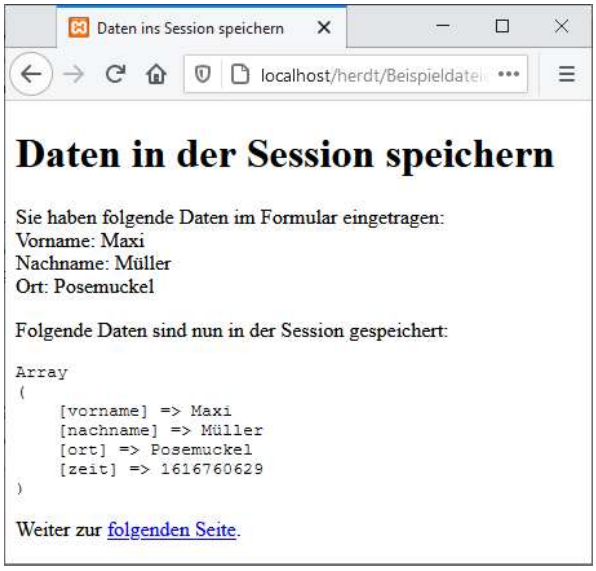
```
<?php
①   session_start();
    echo "<!DOCTYPE html><html>";
    echo "<head><meta charset=\"UTF-8\">";
    echo "<title>Daten ins Session speichern</title></head>";
    echo "<body><h1>Daten in der Session speichern</h1>";
    echo "<p>Sie haben folgende Daten im Formular eingetragen:";
②   echo "<br>Vorname: " . $_POST["vorname"];
    echo "<br>Nachname: " . $_POST["nachname"];
    echo "<br>Ort: " . $_POST["ort"] . "</p>";
③   $_SESSION["vorname"] = $_POST["vorname"];
    $_SESSION["nachname"] = $_POST["nachname"];
    $_SESSION["ort"] = $_POST["ort"];
    $_SESSION["zeit"] = time();
    echo "<p>Folgende Daten sind nun in der Session
          gespeichert:</p>";
    echo "<pre>";
④   print_r($_SESSION);
    echo "</pre>";
    echo "<p>Weiter zur <a href='auslesen.php'>folgenden
          Seite</a>.</p></body></html>";
```

① Die laufende Session wird fortgesetzt.

② Die Daten aus dem übermittelten Formular werden über die Array-Variable `$_POST` ausgegeben.

③ In der Array-Variablen `$_SESSION` werden die Einträge mit den Array-Schlüsseln `vorname`, `name` und `ort` (Daten aus dem Formular) sowie zusätzlich die Variable `zeit`, die den aktuellen Zeitstempel enthält, gespeichert. Damit werden die Angaben in der aktuellen Session-Datei im Ordner *C:\xampp\tmp* (Windows) bzw. */lampp/temp* (Mac) gespeichert und für den Zugriff über die Variable `$_SESSION` während der gesamten Session bereitgestellt.

④ Zur Überprüfung der in der Session gespeicherten Variablen wird mit der Funktion `print_r()` die Variable `$_SESSION` ausgelesen und angezeigt.



Ausgabe der Beispieldatei „auswertung.php“

Session-Datei anzeigen lassen

Session-Variablen können vom Client nicht manipuliert werden. Nachdem sie erzeugt und gespeichert wurden, existieren sie nur im Datenspeicher des Servers und können somit nur vom Skript gelesen werden. PHP liest diese Textdatei am Anfang jeder Session ein. Bei jeder Wertzuweisung an die Session-Variable `$_SESSION` speichert PHP die geänderten oder neu hinzugekommenen Session-Daten wieder ab.

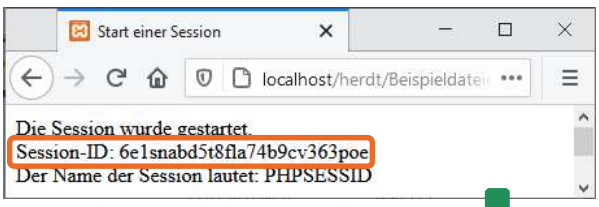
- Wechseln Sie unter Windows in den Ordner *C:\xampp\temp* bzw. */lampp/temp* unter macOS.
- Öffnen Sie die Datei mit der aktuellen Session-ID mit Notepad++ oder einem einfachen Texteditor (unter macOS benötigen Sie root-Rechte, um die Datei zu öffnen). Der Dateiname der Session-Datei besteht aus dem Präfix `sess_` , gefolgt von der Session-ID (z. B. `sess_6e1snabd5t8fla74b9cv363poe`).

  Die Datei hat folgenden Inhalt (in Form eines sogenannten serialisierten Arrays):

```
vorname|s:4:"Maxi";nachname|s:7:"Müller";ort|s:10:"Posemuckel";zeit|
i:1616760629;
```

Die einzelnen Variablen werden nach folgendem Schema abgelegt:

```
Name|Datentyp[:Länge]:Variablen-Inhalt;
```



Anzeige der aktuellen Session-ID



Session-Datei mit gespeicherten Daten

© HERDT-Verlag

11.4 Daten einer Session abrufen

Die gespeicherten Werte einer Session können Sie über den Namen der jeweiligen Variablen (dem Array-Schlüssel in `$_SESSION`) direkt ansprechen bzw. über die Funktion `foreach()` komplett auslesen. Da es sich bei der Variablen `$_SESSION` um eine Array-Variable handelt, können Sie alle Funktionen zur Weiterverarbeitung der Daten anwenden, die Sie aus dem Kapitel *„Arrays“* kennen.

Beispiel: *auslesen.php*

Die Werte, die Sie im vorigen Beispiel in der Session-Datei gespeichert haben, möchten Sie auf einer neuen Seite auslesen. In dem Beispiel werden zwei Möglichkeiten aufgezeigt.

```
<?php
① session_start();
?><!DOCTYPE html>
<html>
  <head><meta charset="UTF-8"><title>Auslesen der
  Session-Daten</title></head>
  <body>
    <h1>Auslesen der Session-Daten</h1>
    <?php
    echo "<p><em>Folgende Variablen wurden gespeichert
         ('foreach'):</em><br>";
② foreach ($_SESSION as $key => $value) {
      echo $key . ": " . $value . "<br>";
    }
    echo "<p><em>Direktes Ansprechen der Variablen:</em><br>";
③ echo "Vorname aus der Session-Datei: " .
          $_SESSION["vorname"] . "<br>";
    echo "Nachname aus der Session-Datei: " .
          $_SESSION["nachname"] . "<br>";
    echo "Ort aus der Session-Datei: " . $_SESSION["ort"] . "<br>";
    echo "Zeitstempel aus der Session-Datei: " .
          $_SESSION["zeit"] . "</p>";
    echo "<p>Weiter zur <a href='session_destroy.php'>folgenden
          Seite</a>.</p>";
    ?>
  </body>
</html>
```

① Die laufende Session wird fortgesetzt.

② Über die `foreach()`-Schleife wird jedes Element des Arrays `$_SESSION` ausgelesen. Mit den Standardvariablen `$key` und `$value` können Sie den Schlüssel und den Wert jeder Session-Variablen am Bildschirm anzeigen lassen.

③ Zusätzlich wird jede Variable einzeln anhand des Namens angesprochen und ausgegeben.



Anzeige der Beispieldatei „auslesen.php“

11.5 Session-Daten und Session löschen

Neben dem Speichern und Auslesen der Daten können Sie diese auch komplett löschen und die Session beenden. Dies ist z. B. notwendig, wenn in einem Online-Shop die Bestellung ausgeführt wurde und somit der Warenkorb des Kunden gelöscht wird.

Löschen der Session-Daten

Wenn Sie alle Session-Variablen löschen wollen, also komplett die Inhalte der Array-Variablen `$_SESSION` sowie den Inhalt der Session-Datei, dann verwenden Sie die folgende Anweisung:

- Mit dieser Anweisung initialisieren Sie die Session-Variable neu und leeren damit das vorhandene Array bzw. Sie ersetzen das bestehende Session-Array durch ein leeres Array. Die Session läuft nach dieser Anweisung weiter.

```
$_SESSION = array();
```

Wollen Sie einzelne Variablen der Session löschen und die restlichen Daten unberührt lassen, verwenden Sie die Funktion `unset()`:

- Sie geben als Parameter `Variable` die genaue Bezeichnung der Variablen an, die Sie löschen möchten.

  Falls Sie aus den vorherigen Beispielen in diesem Kapitel den Vornamen aus der Session entfernen möchten: `unset($_SESSION["vorname"]);`

```
unset(Variable);
```

Löschen der Session mit der Funktion `session-destroy()`

Wenn Sie die komplette Session beenden wollen, reicht es nicht aus, die Session-Daten zu löschen. Die Session läuft weiter. Durch Ausführen der Funktion `session_destroy()` wird die Session-Datei gelöscht. Beim nächsten Versuch, Session-Daten zu speichern, oder beim Versuch, eine Session zu initialisieren, meldet PHP zurück, dass es die dazugehörige Session-Datei nicht (mehr) findet. In diesem Fall wird eine neue Session gestartet. Eine neue Session-ID wird erzeugt und die dazugehörige Session-Datei angelegt.

- Mit `session_destroy()` beenden Sie die aktuelle Session. Dabei wird die Session-Datei im Ordner *C:\xampp\tmp* bzw. */lampp/temp* gelöscht.

```
session_destroy();
```

Beispiel: *session_destroy.php*

Die Daten der bisherigen Session sollen gelöscht und anschließend die gesamte Session beendet werden.

Schulversion
© HERDT-Verlag

```
<?php
①   session_start();
?><!DOCTYPE html>
<html>
  <head><meta charset="UTF-8">
    <title>Session-Daten und Session löschen</title></head>
  <body>
    <h1>Session-Daten und Session löschen</h1>
    <?php
    echo "<pre>";
②   print_r($_SESSION);
    echo "</pre>";
③   unset($_SESSION["vorname"]);
    echo "<pre>";
②   print_r($_SESSION);
    echo "</pre>";
④   $_SESSION = array();
②   print_r($_SESSION);
    echo "<p>Die Session mit der ID " . session_id() .
         " wurde ";
⑤   if (session_destroy()) {
      echo "erfolgreich gelöscht.";
    } else {
      echo "nicht gelöscht.";
    }
    echo "</p>";
    ?>
  </body>
</html>
```

① Die aktuelle Session wird fortgesetzt.

② Mit der Funktion `print_r()` geben Sie zur Kontrolle die Session-Variablen mehrmals am Bildschirm aus. Die Ausgabe des `pre`-Tags vor und hinter dem `print_r()` dient der zeilenweisen Ausgabe im Browser.

③ Mit der Funktion `unset()` löschen Sie die angegebene Session-Variable (`$_SESSION["vorname"]`). Alle anderen Session-Variablen bleiben unberührt. Die Session läuft weiter.

④ Indem Sie der Variablen `$_SESSION` ein leeres Array zuweisen, löschen Sie alle Session-Variablen.



Ausgabe der Beispieldatei „session_destroy.php“

⑤ Danach löschen Sie die Session-Datei und damit die Session. Die Funktion `session_destroy()` liefert einen Wert zurück, und zwar `TRUE`, wenn der Löschvorgang erfolgreich war, ansonsten `FALSE`. Über eine `if`-Abfrage generieren Sie eine entsprechende Ausgabe.

11.6 Fallbeispiel „Shop“

Sie haben in den vorigen Abschnitten die grundlegenden Funktionen rund um Sessions kennengelernt. Die bisherigen Beispiele waren allerdings linear aufgebaut. In typischen Anwendungsbeispielen von Sessions wie z. B. Shops werden Sie jedoch mit weiteren Problemen konfrontiert:

- Sie arbeiten in der Regel innerhalb einer Session mit mehreren Formularen und müssen achtgeben, dass sich in der Session gespeicherte Daten der einzelnen Formulare nicht gegenseitig überschreiben.
- Sie rufen ein Formular eventuell mehrfach auf. Im Idealfall sind bestellte Artikelmengen eingetragen und Sie können jederzeit weitere Artikel bestellen oder Bestellmengen verändern.
- Sie können die Seiten beliebig innerhalb der Session wechseln. Daten dürfen dabei nicht verloren gehen.

Beschreibung des Fallbeispiels

Um die genannten Sachverhalte abzubilden, zeigt das Fallbeispiel einen kleinen Shop mit zwei Formularen (= Artikelgruppen), über welche Sie Artikel bestellen können. Sie können jederzeit zum Warenkorb sowie auf die einzelnen Formularseiten wechseln. Um den Bestellvorgang zu beenden, geben Sie auf einer abschließenden Seite Ihre persönlichen Daten ein. Die Daten zur Bestellung werden ausgegeben und in einer *.csv-Datei gespeichert. Danach wird die Session beendet.

Das Fallbeispiel ist möglichst einfach gehalten. Aus Gründen der Übersichtlichkeit und Nachvollziehbarkeit wurden weder ein ansprechendes Layout noch zusätzliche Funktionen programmiert.

Das Shopbeispiel besteht aus folgenden Dateien:

Dateiname	Kurzbeschreibung
fallbeispiel_start.php	Startseite des Shops, mit Links auf die Formulare zur Bestellung von Schokolade und Pralinen
fallbeispiel_artikel.inc.php	Include-Datei, in der die verwendeten Artikel in Array-Variablen zur Verfügung gestellt werden
fallbeispiel_form-schoko.php *fallbeispiel_form-praline.php*	Formulardateien zur Bestellung von Artikeln. Pro Artikelgruppe wird eine separate Formulardatei verwendet.
fallbeispiel_warenkorb.php	Zentrale Auswertungsdatei für die übermittelten Daten aus den Formularen und dem Warenkorb
fallbeispiel_kasse.php	Datei zum Abschließen des Bestellvorgangs, Eingabe persönlicher Daten, Speichern der Bestellung in einer externen Datei und Beenden der Session

Schulversion
© HERDT-Verlag